

STŘEDNÍ PRŮMYSLOVÁ ŠKOLA MLADÁ BOLESLAV

ROČNÍKOVÁ PRÁCE



Dan Silbernagl

Mladá Boleslav 2025

STŘEDNÍ PRŮMYSLOVÁ ŠKOLA MLADÁ BOLESLAV

ROČNÍKOVÁ PRÁCE

Autor: Dan Silbernagl

Studijní obor: 18-20-M/01 Informační technologie

Vedoucí práce: Jan Till

Mladá Boleslav 2026

Obsah

Contents

Obsah	3
1 Úvod	6
2 Analýza tématu a motivace	7
2.1 Výběr tématu a jeho význam	7
2.2 Cílová skupina a uživatelská očekávání	7
3. Použité technologie	8
3.1 Frontend – UMG, Fab a VR interakce	8
3.1.1 Environment - prostředí	8
3.1.2 Fab	9
3.1.3 Landscape	9
3.2 Backend – Blueprints	9
3.3 Prostředí a scéna – Level Design	10
3.3.1 Prevence kinotézy – Vr Sickness	10
3.4 Nodes	10
3.4.1 Architektura a tok exekuce	11
3.4.2 Implementace binární logiky a stavové řízení	11
3.4.3 Modulární funkce a odezva systému	11
4 Funkcionalita Aplikace	12
4.1 Zbraň	12
4.1.1 SecondaryGrab a stabilizace	13
4.1.2 Munice a Zásobník	13
4.1.3 Vizualní a akustická odezva výstřelu	13
4.1.4 Detekce zasahu a LineTrace	14
4.2 Ruce	14
4.2.1 Princip fungování rukou a jejich propojení s ovladači	14
4.2.2 Grabbing a grabComponent	15
4.2.3 Správa stavů a předávání objektu mezi rukami	15

4.2.4 Animace a Enumerate	15
4.3 Teleportace terče po zásahu	16
4.3.1 Detekce zásahu pomocí LineTraceByChannel	16
4.3.2 SetActorLocation	16
4.3.3 Logika náhodného generování pozic	16
4.4 Bodovací systém	17
4.4.1 AddScore	17
4.4.2 Matematické zpracování pomocí node Add	17
4.4.3 Propojení s uživatelským rozhraním	17
5 Analýza řešení problémů při vývoji interakcí	18
5.1 Selhání exekuce funkce TryGrab	18
5.2 Limity a směr dalšího vývoje	18
6 Závěr	19
6.1 Poděkování	19
7 Přílohy	19
7.1 Zdroje	19

Prohlášení

Prohlašuji, že jsem svou ročníkovou práci vypracoval samostatně a použil jsem pouze podklady (literaturu, projekty, SW atd.) uvedené v přiloženém seznamu.

Nemám závažný důvod proti zpřístupňování této ročníkové práce v souladu se zákonem č. 121/2000 Sb., o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon) v platném znění.

V Mladé Boleslavi dne podpis:



1 Úvod

Smyslem této práce bylo vytvořit funkční aplikaci ve virtuální realitě s názvem VRTrainer, která bude sloužit pro trénink přesnosti, reakce a celkového umu ovladatelnosti a střelby ze zbraně v FPS hrách ve virtuální realitě. Cílem bylo vytvořit funkční Aimtrainer, ve kterém by se uživatel naučil, jak správně ovládat zbraň, zlepšit svou techniku, rychlost a přesnost. Aplikace byla vytvořena pomocí technologie Unreal Engine¹. Celá logika zbraně a rukou byla vytvořena za pomoci nodes² (uzlů) a je silně založena na boolean podmínkách. Boolean podmínka je typ podmínky, kde se Boolean³ nastavuje buď na True, nebo False, a podle toho se určuje, jak bude program pokračovat.

Hlavním důvodem pro realizaci tohoto projektu byla osobní záliba v aimtrainerech, aimtrainingu a hrách tohoto typu. Toto téma je zajímavé jak z pohledu hráče, tak i skrze technické řešení a logiku, která stojí v pozadí a není pro běžného uživatele vidět. Přestože se může téma aimtraineru jevit jako složité a netradiční, i takovéto téma lze zpracovat profesionálně a vytvořit aplikaci, která bude moderní, uživatelsky přívětivá, ale i jednoduchá a přehledná.

¹ Unreal Engine [online]. [cit. 2026-04-22]. Dostupné z: <https://www.unrealengine.com/>

² Blueprints Visual Scripting [online]. [cit. 2026-04-22].
Dostupné z: <https://docs.unrealengine.com/5.3/en-US/blueprints-visual-scripting-in-unreal-engine/>

³ Boolean data type [online]. [cit. 2026-04-22].
Dostupné z: https://en.wikipedia.org/wiki/Boolean_data_type



2 Analýza tématu a motivace

2.1 Výběr tématu a jeho význam

Téma virtuálního aim traineru bylo zvoleno z důvodu dlouhodobé záliby v aim trainerech a také v FPS hrách všeho druhu. Velkou inspirací byly aim trainingové aplikace jako Aimlabs⁴ a Kovaak's⁵. V průběhu ročníkové práce se ukázal vývoj aim traineru ve virtuální realitě jako až příliš složitý a časově náročný úkol.

2.2 Cílová skupina a uživatelsá očekávání

Cílovou skupinu tvoří jednotlivci, kteří chtějí zlepšit své dovednosti ve virtuální realitě v oblasti střelby, přesnosti se zbraní, ovládání zbraní a reakcí na měnící se podmínky pro zasažení cíle. S ohledem na skutečnost, že velká část cílové skupiny nemusí být zručná nebo se může s virtuální realitou teprve seznamovat, byl kladen důraz na jednoduchost a dobrou viditelnost cílů. Pro mnoho uživatelů to může být první zkušenost s virtuální realitou, proto je potřeba zajistit, aby celá aplikace byla jednoduchá a zábavná.

⁴ Aimlabs [online]. [cit. 2026-04-28]. Dostupné z: <https://aimlabs.com/>

⁵ Kovaak's [online]. [cit. 2026-04-28]. Dostupné z: <https://kovaaks.com/>



3. Použité technologie

Pro vývoj aplikace byly využity moderní technologie odpovídající současným trendům a standardům ve vývoji a tvorbě her ve virtuální realitě za pomoci Blueprintů⁶. Projekt byl koncipován s důrazem na vysoký výkon a nízkou latenci. To jsou klíčové parametry pro plynulý uživatelský zážitek z virtuální reality. Celá aplikace je rozdělena do dvou částí: na vizuální (frontend), která zajišťuje vykreslování, herní prostředí, widgety a rozhraní, a na funkční logiku aplikace (backend). Na tu bylo použito vizuální skriptování (Blueprints) a byla stavěna na systému stavových proměnných typu Boolean⁷. Tento přístup umožňuje exaktní řízení rozhodovacích procesů, jako je validace zásahů, zjišťování, zda zbraň vystřelila či nikoliv, kdy mají náboje ze zásobníku zmizet, kdy a jaká animace rukou se má použít, a v neposlední řadě, jaký střelecký režim (shooting mode) je zrovna nastaven. Cílem bylo zvolit takové nástroje, které budou efektivní, modulární, ale také snadno spravovatelné a rozšiřitelné o další funkcionalitu.

3.1 Frontend – UMG, Fab a VR interakce

Pro tvorbu uživatelského rozhraní a vizuálních interakcí byl zvolen systém Unreal Motion Graphics (UMG)⁸. Tento nástroj umožňuje vytvářet komponentově orientované prvky které jsou v prostředí virtuální reality implementovány jako Widget Components⁹. Na rozdíl od klasických aplikací jsou widgety umístěny přímo ve 3D prostoru (world space). To umožňuje hráči přirozenou interakci pomocí VR ovladačů. Systém zajišťuje vysoký výkon při vykreslování a také snadnou správu dynamických prvků, jako je aktuální skóre.

3.1.1 Environment - prostředí

Celé prostředí, ve kterém se může uživatel pohybovat pomocí teleportu, bylo vytvořeno za pomoci staženého balíčku modelů (asset packu) z platformy Fab¹⁰. Inspirací pro celou mapu byl obrázek kresleného středověkého města. Ohraničení města tvoří hradby, které zároveň slouží jako hranice prostoru, do něhož se uživatel může dostat. Hradby byly inspirovány obranným systémem měst z animovaného seriálu Attack on Titan¹¹. Tímto seriálem bylo inspirováno i rozestavení města, které se skládá ze dvou soustředných kruhů a to vnitřního a vnějšího. Styl středověkého (medieval) města byl zvolen z důvodu jednoduchosti a neutrálních barev, díky kterým uživatel snadno rozezná terče od pozadí. V

⁶ Blueprint Programming Basics [online]. [cit. 2026-04-28]. Dostupné z:

<https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprint-basics-in-unreal-engine>

⁷ Variables in Blueprints [online]. [cit. 2026-04-28]. Dostupné z:

<https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprint-variables-in-unreal-engine>

⁸ UMG UI Designer [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/umg-ui-designer-in-unreal-engine>

⁹ Widget Components [online]. [cit. 2026-04-28]. Dostupné z:

<https://dev.epicgames.com/documentation/en-us/unreal-engine/widget-components-in-unreal-engine>

¹⁰ Fab Marketplace [online]. [cit. 2026-04-28]. Dostupné z: <https://www.fab.com/>

¹¹ Attack on Titan (Shingeki no Kyojin) [online]. [cit. 2026-04-28]. Dostupné z: <https://shingeki.tv/>



samotném středu města se nachází malé náměstí, na kterém se odehrávají veškeré funkce aim traineru. Uživatel je po spuštění umístěn (spawnut) do středu náměstí, přičemž jeho pohled směřuje přímo na terče. Ty byly pro lepší viditelnost a odhad vzdálenosti zvýrazněny svítivou modrou barvou, která v kontrastu s pozadím perfektně vyniká od pozadí města. Na stolku vedle uživatele jsou položeny tři zásobníky a zbraň, která neobsahuje zásobník.

3.1.2 Fab

Fab¹² je aplikace integrovaná v rámci Unreal Engine, která slouží pro sdílení, stahování a nákup nebo prodej asset packů, textur a meshů. Pokud si tvůrce prostředí (environmentu) nechce vše tvořit sám, představuje Fab nejlepší dostupnou možnost. Fab obsahuje obrovské množství assetů zdarma, od textur trávy s gradientem podle výšky a sklonu až po rozsáhlé balíčky (asset packy) všeho druhu. Jediné, na co si musí designér dát pozor, je počet polygonů použitých ve stažených assetech, a to z důvodu možného snížení výkonu hry.

3.1.3 Landscape

Tvorba landscapeu se v Unreal Enginu dá realizovat dvěma způsoby. První možností je, že si designér může stáhnout takzvanou heightmapu¹³. Unreal Engine umí sám rozpoznat výškové rozdíly z heightmapy a podle ní vytvořit landscape. Druhou možností je, že si designér může vytvořit svůj vlastní landscape pomocí samotné funkce v Unreal Enginu¹⁴. Tento projekt byl tvořen pomocí stažené heightmapy, která byla postupně upravována, aby se shodovala s představou ostrova s městem na pobřeží oceánu.

3.2 Backend – Blueprints

Vnitřní logika aplikace je postavena na systému Blueprints Visual Scripting. Tento přístup umožňuje efektivně propojovat události (events) s konkrétními akcemi bez jakékoli nutnosti psaní textového kódu. Jedním z klíčových prvků je využití binární logiky (booleovských proměnných), ty řídí veškeré větvení programu. Díky Booleanům¹⁵ je zajištěna velmi přesná validace herních stavů, jako je ověření, zda je zbraň nabitá, co zrovna

¹² Using Fab in Unreal Engine [online]. [cit. 2026-04-28]. Dostupné z:

<https://dev.epicgames.com/documentation/en-us/unreal-engine/fab-integration-in-unreal-engine>

¹³ Importing Heightmaps [online]. [cit. 2026-04-28]. Dostupné z:

<https://dev.epicgames.com/documentation/en-us/unreal-engine/importing-landscapes-in-unreal-engine>

¹⁴ Landscape Tool [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/landscape-outdoor-terrain-in-unreal-engine>

¹⁵ Boolean Logic in Games [online]. [cit. 2026-04-28]. Dostupné z: <https://www.geeksforgeeks.org/boolean-logic-in-game-development/>



ruka drží a jaká animace rukou se má v daný moment přehrát. Dále proměnné také určují, zda zbraň vystřelila, jakou část zbraně ruka zrovna drží a zda je aktivní sekundární úchop (Secondary Grab).

3.3 Prostředí a scéna – Level Design

Pro účely aim traineru bylo vytvořeno specifické herní prostředí v podobě středověkého městečka s hradbami. Toto prostředí plní jak estetickou funkci, tak pomáhá s prostorovou orientací ve virtuální realitě. Hranice herního prostoru byly definovány pomocí hradeb, které jsou poskládány do kruhového tvaru po obvodu města. Při tvorbě byl kladen důraz na optimalizaci scény, aby bylo dosaženo stabilní snímkové frekvence (FPS¹⁶). Pokud by měla aplikace ve výsledku nízkou snímkovou frekvenci, zvyšuje se při dynamických scénách u uživatele šance na vznik kinetózy¹⁷.

3.3.1 Prevence kinetózy – Vr Sickness

Plynulost zobrazení je ve virtuální realitě kritickým faktorem. Pokud bude mít aplikace nízkou snímkovou frekvenci nebo bude docházet k náhlým propadům průměrné snímkové frekvence, zvyšuje se u uživatele riziko na vznik kinetózy. Tento typ nevolnosti je způsoben konfliktem mezi zrakovým vjemem a statickým stavem těla uživatele.

3.4 Nodes

Celý systém je postaven na vizuálním skriptování, lépe známém jako Blueprints. Ten v Unreal Engineu nahrazuje klasické psaní kódu. Všechna interaktivita, od pohybu hráče až po komplexní mechaniky střelby, byla realizována pomocí Nodes (uzlů). Tento systém funguje na principu grafického propojování jednotlivých bloků, které reprezentují specifické funkce, proměnné, skripty, animace a sound cues¹⁸.

¹⁶ FPS (Frames Per Second) [online]. [cit. 2026-04-28]. Dostupné z: https://en.wikipedia.org/wiki/Frame_rate

¹⁷ VR Sickness and Kinetosis [online]. [cit. 2026-04-28]. Dostupné z: <https://www.oculus.com/blog/understanding-and-reducing-vr-sickness/>

¹⁸ Sound Cues in Unreal [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/sound-cue-editor-in-unreal-engine>



3.4.1 Architektura a tok exekuce¹⁹

Logika celé aplikace je postavena na objektově orientovaném přístupu (OOP²⁰), kde každý node reprezentuje specifickou operaci. Systém zpracovává vstupy z ovladačů virtuální reality a poté je převádí na konkrétní herní události. Díky těmto výstupům z ovladače je program schopen přesně určit, jestli ruka drží v ruce zbraň, zásobník, slider, nebo třeba nedrží nic. Tento tok také obsahuje pokročilé řízení animací na základě dynamického Vyhodnocení stavu postavy, a na jakém základě plynule přepíná mezi jednotlivými sekvencemi rukou a těla. To zajišťuje vysokou úroveň vizuální věrnosti.

3.4.2 Implementace binární logiky a stavové řízení

Vzhledem k tomu, že backend aplikace je silně založen na booleovských podmínkách, hrají v uzlových grafech velkou roli rozhodovací uzly (Branch²¹). Tyto uzly fungují na principu vyhodnocování podmínky. Branch vyhodnotí připojenou Boolean proměnnou a určí, zda je nastavena na True nebo False. Podle toho se určí, jak bude program pokračovat.

3.4.3 Modulární funkce a odezva systému

Použití Blueprintů nám umožňuje vytvářet samostatné logické celky a funkce, které se spouštějí jen při určité události. To zahrnuje vše od výpočtu zásahu cíle, jakou animaci má zrovna v daný moment program přehrát, až po jaký zvuk má zbraň udělat. Toto řešení je díky tomu nejvíce přehledné a umožňuje snadné přidání dalších funkcí nebo změn chování zbraní a cílů v budoucnosti aplikace.

¹⁹ Execution Flow in Blueprints [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/execution-flow-in-unreal-engine-blueprints>

²⁰ Object-Oriented Programming (OOP) [online]. [cit. 2026-04-28]. Dostupné z: https://en.wikipedia.org/wiki/Object-oriented_programming

²¹ Branch Nodes [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/flow-control-in-unreal-engine-blueprints#branch>



4 Funkcionalita Aplikace

4.1 Zbraň

Zbraň je navržena jako Actor Blueprint, který má za úkol co nejbližší simulaci reálného chování pistole. Velmi důležitou funkcí pro stabilizaci a přesnost střelby je SecondaryGrab²² (sekundární úchop). Ta umožňuje hráči držet zbraň oběma rukama a zároveň s ní přirozeně a přesně mířit na cíl. Součástí modelu zbraně jsou i interaktivní prvky, jako je slider a vyjímatelný zásobník. Logika mizení munice a vyprázdnění zásobníku je řešena skrze proměnné typu Integer, které se starají o limit počtu výstřelů a mizení nábojnic po vystřelení. Pro zvýšení herního zážitku jsou ve zbrani implementovány Niagara Systems²³, ty se starají o vizuální efekty záblesku u ústí hlavně po výstřelu a o padání vystřelených nábojů ze slideru. U procesu střelby jsou také použity prostorové zvuky střelby, přebití, slideru a hitu.

²² Secondary Grab Logic in VR [online]. [cit. 2026-04-28]. Dostupné z:

<https://dev.epicgames.com/documentation/en-us/unreal-engine/xr-interaction-setup-in-unreal-engine>

²³ Niagara Visual Effects [online]. [cit. 2026-04-28]. Dostupné z:

<https://dev.epicgames.com/documentation/en-us/unreal-engine/niagara-visual-effects-in-unreal-engine>



4.1.1 SecondaryGrab a stabilizace

Pro dosažení maximální přesnosti při střelbě Aim Trainer využívá SecondaryGrab. Tato funkce je zajištěna skrze dva body úchopu. V momentě, kdy systém detekuje interakci druhého ovladače s daným socketem pro SecondaryGrab, dojde k přepnutí režimu ze SingleGrab na SecondaryGrab. V momentě, kdy se začne používat SecondaryGrab, se orientace zbraně přestane řídit jedním bodem a začne se dynamicky měnit podle polohy obou ovladačů v prostoru. Zbraň se sama automaticky srovná do přímky mezi ruce uživatele, což eliminuje chvění zbraně, které je typické pro držení jednou rukou, a zlepší se jak přesnost, tak i recoil zbraně.

4.1.2 Munice a Zásobník

Logika správy munice používá oddělený objekt zásobníku a celočíselnou proměnnou index. Po zasunutí zásobníku do zbraně dojde k aktivaci události²⁴, ta nastaví aktuální stav nábojů na maximální hodnotu. Každý výstřel následně tuto hodnotu sníží o jednu jednotku. V momentě, kdy proměnná index dosáhne nuly, systém skrze Branch deaktivuje funkci střelby. Pro pokračování ve střelbě je nutné zásobník fyzicky vyjmout a nahradit ho novým.

4.1.3 Vizualní a akustická odezva výstřelu

Každý výstřel je okamžitě doprovázen odezvou, tu zajišťují částicové systémy Niagara. Niagara systémy generují vizuální efekt záblesku z ústí hlavně a simulují vypadávání prázdných nábojnic ze slideru zbraně. Tyto efekty jsou v synchronizaci s přehráváním zvuku výstřelu v místě zbraně. Tato kombinace vizuálních a sluchových vjemů je klíčová pro tzv. imerzi²⁵.

²⁴ Blueprint Events [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/events-in-unreal-engine-blueprints>

²⁵ Immersion in Virtual Reality [online]. [cit. 2026-04-28]. Dostupné z: [https://en.wikipedia.org/wiki/Immersion_\(virtual_reality\)](https://en.wikipedia.org/wiki/Immersion_(virtual_reality))



4.1.4 Detekce zásahu a LineTrace

Pro vyhodnocování střelby je použita metoda trasování neviditelného paprsku, neboli LineTraceByChannel²⁶. Ve stejný moment, kdy uživatel stiskne spoušť zbraně, z ní mimo výstřel aplikace vyšle i neviditelný paprsek, který testuje kolize s okolním prostředím. Pokud paprsek projde přes kolizní²⁷ zónu terče, systém okamžitě získá data o zásahu. Tento přístup byl zvolen pro svou extrémní rychlost, efektivitu a přesnost, což zajišťuje, že aplikace běží bez jakéhokoli zpoždění

4.2 Ruce

Systém virtuálních rukou představuje v rámci tohoto projektu jediný způsob interakce mezi uživatelem a digitálním prostředím. Ruce nebyly tvořeny pouze jako statické modely, ale jako funkční celky. Ty v reálném čase zpracovávají data z ovladačů a transformují je na pohyby a další různé úkony ve virtuálním prostředí

4.2.1 Princip fungování rukou a jejich propojení s ovladači

Aby se ruce ve virtuální realitě hýbaly přesně tak, jak se hýbají v daný moment ruce uživatele, jsou v kódu připojeny na komponenty MotionControllerLeftGrip a MotionControllerRightGrip. Tyto komponenty fungují jako neviditelné body, které neustále sledují, v jaké pozici se v reálném světě ovladače v rukou uživatele nacházejí. Informace o přesné poloze ovladačů se přenášejí na 3D modely rukou (Skeletal Mesh²⁸), ty hráč vidí před sebou ve virtuální realitě. Výsledný pohyb rukou tak díky nekonečné aktualizaci polohy rukou vypadá velmi přirozeně a plynule. Vysoká přesnost je nezbytně nutná nejen pro přesnost míření se zbraní, ale i pro pohodlí uživatele.

²⁶ Line Trace By Channel [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/using-a-line-trace-by-channel-in-unreal-engine>

²⁷ Collision Channels [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/collision-in-unreal-engine>

²⁸ Skeletal Mesh Components [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/skeletal-mesh-actors-in-unreal-engine>



4.2.2 Grabbing a grabComponent

Schopnost ruky interagovat se zbraní nebo zásobníkem je možná díky komponentě GrabComponent. Tato komponenta slouží jako rozhraní pro všechny uchopitelné předměty, které aplikace obsahuje. Ke spuštění GrabComponenty dojde v momentě, kdy je aktivována vstupní akce IA_Grab_Left nebo IA_Grab_Right. Systém potom reaguje spuštěním funkce GetGrabComponentNearMotionController. Poté je spuštěna funkce TryGrab, která zjistí, zda ruka drží nějaký předmět, a pokud ano, tak ho ruka chytne. Funkce TryRelease je přesným opakem funkce TryGrab; TryRelease zajišťuje uvolnění objektu, který ruka drží.

4.2.3 Správa stavů a předávání objektu mezi rukami

Aby aplikace dokázala správně rozpoznat, která ruka manipuluje s konkrétním objektem, využívá Blueprint systém s referenčními podmínkami HeldComponentLeft a HeldComponentRight. Po úspěšném spuštění funkce TryGrab se do referenčních podmínek uloží GrabComponent. V případě, že systém detekuje pokus o úchop objektu druhou rukou, než tou, která ho již drží, provede se automatické vyčištění reference v odpovídající referenční proměnné a teprve poté se zahájí nový proces úchopu. Tento systém, podporovaný neustálými kontrolami stavu za pomoci Boolean proměnné IsHeld, eliminuje problémy v transformaci a umožňuje přirozenou manipulaci.

4.2.4 Animace a Enumerate

Pohyb rukou a jejich prstů je řízen pomocí Animation Blueprintu, který využívá seznam výčtových typů, neboli Enumerate²⁹. Tyto výčtové typy definují různé stavy, ve kterých se animace³⁰ rukou má zrovna nacházet. Pomocí aktuální hodnoty v Enumeratu dochází k automatickému výběru odpovídající animace rukou. Tento systém je také schopen dynamicky reagovat na tvar drženého předmětu, čímž zajišťuje, že se prsty s modelem objektu vizuálně nepřekrývají.

²⁹ Enums in Blueprints [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/enum-nodes-in-unreal-engine-blueprints>

³⁰ Animation Blueprints [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/animation-blueprints-in-unreal-engine>



4.3 Teleportace terče po zásahu

Aby aplikace plnila svou funkci Aim Traineru, je potřeba, aby byla schopna v reálném čase vyhodnocovat přesnost střelby a nějakým způsobem na ni reagovat. Celý tento proces byl navržen takovým způsobem, aby hráč získal okamžitou zpětnou vazbu po každém jednom výstřelu. Systém se stará o to, aby byl každý výstřel správně vyhodnocen a aby se herní prostředí v reakci na vyhodnocení výstřelu měnilo.

4.3.1 Detekce zásahu pomocí LineTraceByChannel

V momentě, kdy hráč stiskne spoušť zbraně a vystřelí, systém využívá metodu zvanou LineTraceByChannel. Tento paprsek letí z hlavní zbraně ve směru míření přesně v ten stejný moment jako vystřelený náboj. Program díky němu okamžitě zjistí, zda tento paprsek cestou narazil do nějakého objektu, který má kolizi. Pokud paprsek protne kolizi od terče, systém okamžitě získá data o hitu a spustí řetězec funkcí. Tato metoda byla zvolena kvůli svojí naprosté přesnosti a nulové náročnosti na výkon.

4.3.2 SetActorLocation

V momentě, kdy je zásah potvrzen, kulička (terč) se nezničí, ale dojde k její teleportaci na nové náhodné místo po osách X a Y. V Blueprintu je k tomu použita funkce SetActorLocation³¹. Místo toho, aby byla kulička smazaná a znovu načtená do paměti počítače, se jí pouze během pár milisekund přepíše dané souřadnice v prostoru. Tento postup byl zvolen, protože výrazně šetří výkon.

4.3.3 Logika náhodného generování pozic

Nová poloha kuličky v aréně není nijak předem určená. Je vypočítána po každém zásahu znovu. Systém k tomu využívá generátor náhodných čísel pro osy X a Y, osa Z je nastavena na konstantní hodnotu. Tím je zajištěno, že se kulička může objevit pouze v různé výšce a směru, ale ne v jiné hloubce. Celý systém se odehrává v Blueprintu BP_kulicka. Tato nepředvídatelnost nutí uživatele neustále měnit polohu a míření zbraně a trénovat tak reakce na změnu podmínek a rychlou koordinaci ruky a oka. Tato teleportace byla inspirována úkolem z FPS Aim Traineru Aimlabs, konkrétně úkolem Gridshot.

³¹ Set Actor Location [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprint-nodes-for-transforming-actors>



4.4 Bodovací systém

Součástí každého Aim Traineru, která nesmí nikdy chybět, je systém hodnocení uživatele. To hráči poskytuje přehled o jeho pokroku a přesnosti. V této aplikaci byl bodovací systém navržen takovým způsobem, aby byl okamžitý, přesný a motivující. Celá jeho logika byla postavena velmi jednoduše, ale efektivně. Za každý zásah terče obdrží uživatel 75 bodů, ty se následně sčítají s celkovým skóre.

4.4.1 AddScore

Jádrem celé logiky je Custom Event³² (uživatelská událost) s názvem AddScore. AddScore čeká na signál od terče, že dostal hit a byl teleportován. V momentě, kdy AddScore dostane daný signál po zásahu kuličky, spustí proces výpočtu bodů a zajistí, aby byly body správně přičteny. Díky tomu, že je tato logika úplně oddělená od zbytku kódu v BP_kulicka, je možné v budoucnu přidávat různé počty skóre a jejich násobky pro jiné terče.

4.4.2 Matematické zpracování pomocí node Add

V momentě, kdy AddScore dostane informaci o tom, že má přičíst body do skóre, se spustí matematická část kódu s uzlem Add (Integer). Tento node sčítá dvě hodnoty dohromady a vrací jejich součet. Výhodou tohoto řešení je jeho spolehlivost, rychlost a efektivita.

4.4.3 Propojení s uživatelským rozhraním

Úplně posledním krokem v logice bodovacího systému je předání aktualizované hodnoty hráči. V momentě, kdy dojde k uložení nového skóre, systém vyšle nové skóre do Widgetu³³ (grafického rozhraní). Díky tomu se číslo na widgetu okamžitě změní. Tato změna probíhá v reálném čase, takže hráč má neustálý přehled o svém skóre.

³² Custom Events [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/create-custom-events-in-unreal-engine-blueprints>

³³ Creating Widgets [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/creating-widgets-in-unreal-engine>



5 Analýza řešení problémů při vývoji interakcí

Pro ověření, jestli je kód funkční, byl implementován diagnostický nástroj Debug Sphere³⁴ (ladicí sféra). Tato sféra je v herním světě přítomna neustále a aktivuje se pouze v momentě, kdy uživatel stiskne tlačítko pro úchop předmětu. Pokud ruka spustí GrabComponent a ten nemá co držet, objeví se okolo ruky červený kruh, ale pokud GrabComponent má co držet, objeví se okolo ruky kruh zelený. Následné testování potvrdilo, že funkce GetGrabComponentNearMotionController se správně spustí a identifikuje Collision Capsule (kolizní kapsli) na rukojeti zbraně. Problém však nastane vzápětí. Poté, co se okolo ruky objeví zelený kruh, se zbraň fyzicky nepřichytí k modelu rukou.

5.1 Selhání exekuce funkce TryGrab

I přesto, že sféra po stisku tlačítka zezelená a potvrdí nález kolizní kapsle, proces se zastavuje uvnitř funkce TryGrab. Úkolem této funkce je převzít identifikovaný objekt a fyzicky ho přichytit k ruce, to se ale v aktuálním stavu aplikace nestane. Problém spočívá v tom, že funkce není schopna vykonat svůj vnitřní řetězec funkcí a událostí pro uchycení objektu. Z důvodu toho, že TryGrab selže dříve, než stačí vykonat poslední část kódu pro úchop objektu, nedojde k žádnému pohybu zbraně a ruka se uzavře naprázdno.

5.2 Limity a směr dalšího vývoje

Aktuální stav aplikace, kdy systém po manuálním sepnutí spustí funkci TryGrab, ale není ji schopen dokončit, definuje jasný technologický limit současné verze aplikace. Pro vyřešení tohoto problému je nezbytné provést podrobnou kontrolu kódu a logiky funkce TryGrab a komponent GrabComponent a GetGrabComponentNearMotionController. Následně je třeba zjistit, proč nedochází k finálnímu propojení mezi komponentou úchopu a ovladačem. Tato analýza jasně vymezila prostor, ve kterém dochází k logickému přerušení, a tudíž i chybě v logice aplikace, což je zásadní pro budoucí opravy a spuštění plně funkční verze Aim Traineru ve virtuální realitě.

³⁴ Drawing Debug Shapes [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/drawing-debug-shapes-in-unreal-engine>



6 Závěr

Cílem této práce bylo navrhnout a vytvořit funkční aplikaci ve virtuální realitě pod názvem VRTrainer, sloužící jako Aim Trainer. Projekt se zaměřil na realistické zpracování manipulace se zbraní, na detekci zásahu a správu terčů v prostředí vytvořeném v Unreal Engine 5. Během vývoje byl kladen důraz především na optimalizaci rukou a na jednoduché, ale názorné ovládání.

Práce na tomto projektu pro mě byla velmi přínosná, hlavně v oblasti vizuálního skriptování pomocí Blueprintů. I přes všechny překážky, které se vyskytly u náročnějších interakcí, mě vývoj ve virtuálním prostředí velmi zaujal a bavil. Tato oblast nabízí neskutečně velký prostor pro další vývoj, jako je například implementace pokročilejší balistiky, přidání nových modelů zbraní nebo rozšiřování tréninkových scénářů. Velké poděkování také náleží Střední průmyslové škole Mladá Boleslav za vytvoření podmínek pro vznik tohoto projektu a za možnost pracovat s moderními technologiemi.

6.1 Poděkování

Poděkování patří také mému vedoucímu projektu, Janu Tillovi, za průběžnou konzultaci se zpracováním jak praktické, tak i písemné části. Jeho rady a zpětná vazba přispěly ke zlepšení kvality výsledného projektu.

7 Přílohy

7.1 Zdroje

Unreal Engine [online]. [cit. 2026-04-22]. Dostupné z: <https://www.unrealengine.com/>

Blueprints Visual Scripting [online]. [cit. 2026-04-22]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprints-visual-scripting-in-unreal-engine/>

Boolean data type [online]. [cit. 2026-04-22]. Dostupné z: https://en.wikipedia.org/wiki/Boolean_data_type

Aimlabs [online]. [cit. 2026-04-28]. Dostupné z: <https://aimlabs.com/>

Kovaak's [online]. [cit. 2026-04-28]. Dostupné z: <https://kovaaks.com/>



Blueprint Programming Basics [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprint-basics-in-unreal-engine>

Variables in Blueprints [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprint-variables-in-unreal-engine>

UMG UI Designer [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/umg-ui-designer-in-unreal-engine>

Widget Components [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/widget-components-in-unreal-engine>

Fab Marketplace [online]. [cit. 2026-04-28]. Dostupné z: <https://www.fab.com/>

Attack on Titan (Shingeki no Kyojin) [online]. [cit. 2026-04-28]. Dostupné z: <https://shingeki.tv/>

Using Fab in Unreal Engine [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/fab-integration-in-unreal-engine>

Importing Heightmaps [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/importing-landscapes-in-unreal-engine>

Landscape Tool [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/landscape-outdoor-terrain-in-unreal-engine>

Boolean Logic in Games [online]. [cit. 2026-04-28]. Dostupné z: <https://www.geeksforgeeks.org/boolean-logic-in-game-development/>

FPS (Frames Per Second) [online]. [cit. 2026-04-28]. Dostupné z: https://en.wikipedia.org/wiki/Frame_rate

VR Sickness and Kinetosis [online]. [cit. 2026-04-28]. Dostupné z: <https://www.oculus.com/blog/understanding-and-reducing-vr-sickness/>

Sound Cues in Unreal [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/sound-cue-editor-in-unreal-engine>



Execution Flow in Blueprints [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/execution-flow-in-unreal-engine-blueprints>

Object-Oriented Programming (OOP) [online]. [cit. 2026-04-28]. Dostupné z: https://en.wikipedia.org/wiki/Object-oriented_programming

Branch Nodes [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/flow-control-in-unreal-engine-blueprints#branch>

Secondary Grab Logic in VR [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/xr-interaction-setup-in-unreal-engine>

Niagara Visual Effects [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/niagara-visual-effects-in-unreal-engine>

Blueprint Events [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/events-in-unreal-engine-blueprints>

Immersion in Virtual Reality [online]. [cit. 2026-04-28]. Dostupné z: [https://en.wikipedia.org/wiki/Immersion_\(virtual_reality\)](https://en.wikipedia.org/wiki/Immersion_(virtual_reality))

Line Trace By Channel [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/using-a-line-trace-by-channel-in-unreal-engine>

Collision Channels [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/collision-in-unreal-engine>

Skeletal Mesh Components [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/skeletal-mesh-actors-in-unreal-engine>

Enums in Blueprints [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/enum-nodes-in-unreal-engine-blueprints>

Animation Blueprints [online]. [cit. 2026-04-28]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/animation-blueprints-in-unreal-engine>



Set Actor Location [online]. [cit. 2026-04-28]. Dostupné z:
<https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprint-nodes-for-transforming-actors>

Custom Events [online]. [cit. 2026-04-28]. Dostupné z:
<https://dev.epicgames.com/documentation/en-us/unreal-engine/create-custom-events-in-unreal-engine-blueprints>

Creating Widgets [online]. [cit. 2026-04-28]. Dostupné z:
<https://dev.epicgames.com/documentation/en-us/unreal-engine/creating-widgets-in-unreal-engine>

Drawing Debug Shapes [online]. [cit. 2026-04-28]. Dostupné z:
<https://dev.epicgames.com/documentation/en-us/unreal-engine/drawing-debug-shapes-in-unreal-engine>